ШИНЖЛЭХ УХААН ТЕХНОЛОГИЙН ИХ СУРГУУЛЬ
МЭДЭЭЛЭЛ, ХОЛБООНЫ ТЕХНОЛОГИЙН СУРГУУЛЬ



БИЕ ДААЛТ №1
ТАЙЛАН

Үйлдлийн систем /F.CSM302/

Сэдэв:	13 – Banker's Algorithm. Судалгаа, хэрэгжилт, туршилтын тайлан
Сурах бичиг:	Operating System Concepts, 8-р бүлэг (Deadlocks)
Хуваарилалтын код:	IV-2-13
Хичээл заасан багш:	Г.Ганбат /F.CS10/
Гүйцэтгэсэн баг:	У.Анужин /B241940071/ Г.Лувсанжал /B242270016/ Г.Ганхүү /B210930028/ Г.Ганжаргал /B222270701/ Б.Лхагвадорж /B231940136/ Я.Төгөлдөр /B242270806/
Огноо:	Улаанбаатар хот, 2026 он

ХУРААНГУЙ

Асуудал. Түгжрэл (deadlock) нь олон процесс хязгаарлагдмал нөөцийг өрсөлдөн авах үед үүсдэг үйлдлийн системийн сонгодог асуудал. Banker's Algorithm нь нөөц олгохын өмнө «олгосны дараа бүх процесс ямар нэг дарааллаар дуусаж чадах уу?» гэж урьдчилан шалгаж, систем үргэлж safe төлөвт байхыг баталгаажуулан түгжрэлээс сэргийлдэг.

Шийдэл. Safety algorithm ба Resource-Request algorithm-ыг Python дээр UI-аас тусдаа цөм (`core.py`) болгон хэрэгжүүлж, сурах бичгийн төслийн шаардлагын дагуу командын мөрийн программ (`cli.py`: `RQ`, `RL`, `*`) болон вэб интерфэйс (HTTP сервер + HTML/JS) бүтээв. Багшийн заасан Resource graph дүрслэл нь процесс ба нөөцийн төрлийн холбоог SVG граф хэлбэрээр зурж, Safe sequence-ийг «▶ Эхлүүлэх» товчоор алхам алхмаар тоглуулна.

Туршилт. 5 файлд нийт 64 автомат тест (цөм 15, CLI 24, CLI-ийн төлөв хамгаалалт 7, HTTP сервер 13, Banker унтраалттай горим 5), номын жишээ, Unsafe, 10×5 том сценари, оролтын алдааны тохиолдлуудыг UI дээр бодитоор туршиж screenshot-оор баримтжуулав.

Гол үр дүн. 64/64 тест тэнцэв. Номын жишээнд эхний төлөв SAFE, хүсэлтийн дараалал (P1 олгоно → P4 хүлээнэ → P0 татгалзана) номын хариутай нийцэв. Цөмийн safe/unsafe дүгнэлтийг 200 санамсаргүй жижиг төлөв дээр бүх сэлгэмэлийг туршдаг (brute-force) бие даасан шалгагчтай тулгасан. UI дээр Allocation, Max, Available, хүсэлтийг өөрчилж дахин турших, Banker-ыг унтрааж түгжрэлд хүрэхийг харьцуулах боломжтой.

Хязгаарлалт (товч). Сервер нэг хэрэглэгчийн төлөвтэй (санах ойд), олон процессыг зэрэг ажиллуулсан (multi-thread) customer загвар хэрэгжээгүй. Дэлгэрэнгүйг 9-р бүлгээс үзнэ үү.

ГАРЧИГ

Хураангуй 1

1. *Удиртгал* 4
- 1.1 Зорилго 4
- 1.2 Ач холбогдол 4
- 1.3 Хамрах хүрээ 4

2. *Сурах бичгийн шаардлага* 4

3. *Онолын үндэс* 5
- 3.1 Түгжрэл ба Banker's Algorithm 5
- 3.2 Өгөгдлийн бүтэц 5
- 3.3 Safety algorithm 6
- 3.4 Resource-Request algorithm 6
- 3.5 SAFE, UNSAFE, DEADLOCK 6
- 3.6 Гараар бодсон номын жишээ 6

4. *Системийн зохиомж* 7
- 4.1 JSON гэрээ (core → UI) 8
- 4.2 HTTP API 8

5. *Хэрэгжилт* 8
- 5.1 Хэл, орчин, сан 8
- 5.2 Цөм (core.py) 9
- 5.3 CLI (cli.py) 9
- 5.4 Вэб UI 9
- 5.5 Бэлэн код, сан, AI-ийн хэрэглээ 10
- 5.6 GitHub 10

6. *Туршилт ба UI дүрслэл* 10
- 6.1 Юуг өөрчилж турших вэ 10
- 6.2 Юуг ажиглах вэ 10
- 6.3 Туршилтын сценарууд 10

7. *Туршилтын арга* 11

8. *Үр дүн* 11
- 8.1 Автомат тестийн үр дүн 11
- 8.2 Номын жишээний дараалсан хүсэлтүүд (бодит гаралт) 12
- 8.3 Бодит screenshot (UI) 13
- 8.4 Нэмэлт туршилтын дүгнэлт 18

9. *Хэлэлцүүлэг* 19
- 9.1 Үр дүнгийн тайлбар 19
- 9.2 Хязгаарлалт ба мэдэгдэж буй асуудлууд 19

10. *Дүгнэлт* 19

Ашигласан материал 20

Хавсралт А. Ажлын төлөвлөгөө 21

Тестийн төлөвлөгөө 21

Хавсралт Б. Нэмэлт орчинд шалгасан тэмдэглэл 23

1. УДИРТГАЛ

1.1 Зорилго

Сурах бичгийн 8-р бүлгийн Programming Projects → Banker's Algorithm төслийг бүрэн хэрэгжүүлэх, түүний ажиллагааг хэрэглэгч алхам алхмаар харж, өөр өгөгдөл, нөхцөл, параметрээр дахин турших боломжтой интерактив дүрслэлтэй болгох, хэрэгжилт ба туршилтын үр дүнг баримтжуулах.

1.2 Ач холбогдол

Түгжрэлээс сэргийлэх алгоритмыг зөвхөн тоон хүснэгтээр гараар бодохоос гадна «яагаад энэ хүсэлтийг олгохгүй байна вэ?» гэдгийг процесс ба нөөцийн граф, Work векторын өөрчлөлтөөр харах нь ойлголтыг ихээхэн хөнгөвчилдөг. Программ нь SAFE, UNSAFE, DEADLOCK гэсэн гурван ойлголтын ялгааг (UNSAFE нь заавал DEADLOCK биш) бодит жишээгээр харуулдаг.

1.3 Хамрах хүрээ

Safety algorithm, Resource-Request algorithm (rollback-тай), нөөц чөлөөлөх (release), Banker-ыг унтрааж харьцуулах горим, CLI (`RQ`/`RL`/`*`), вэб UI (өгөгдөл засах хүснэгт, хүсэлтийн форм, Resource graph, Safe sequence харуулалт, хүсэлтийн түүх), автомат тест, Manual.

2. СУРАХ БИЧГИЙН ШААРДЛАГА

Operating System Concepts (Silberschatz, Galvin, Gagne, 2018) 8-р бүлгийн Banker's Algorithm төслийн шаардлага ба Бие даалт 1-ийн Guide-ын нийтлэг шаардлагыг хэрэгжилттэй харьцуулав.

Хүснэгт 1. Шаардлага ба хэрэгжилт

№	Шаардлага	Хэрэгжилт	Төлөв
1	n процесс, m төрлийн нөөц; тоог эх кодод хатуу тогтоохгүй	`Banker(available, maximum, allocation)` — хэмжээсийг оролтоос авна; UI-д +/− Процесс, Нөөц	Хэрэгжсэн
2	Available-ийг командын мөрийн аргументаар өгөх	`python src/cli.py 10 5 7 --max ...` (нийт нөөц; Allocation 0 тул Available = нийт)	Хэрэгжсэн
3	Max-ийг файлаас унших	`--max examples/book_max.txt` (мөр = процесс, таслал/хоосон зайгаар)	Хэрэгжсэн
4	Allocation эхэндээ 0; Need = Max − Allocation	Allocation өгөөгүй бол 0; `Banker.need` тооцогдоно. Нэмэлтээр `--allocation` файл	Хэрэгжсэн
5	`RQ` — нөөц хүсэх; `RL` — чөлөөлөх; * — матрицуудыг хэвлэх	`cli.py` дахь командууд (+ `exit`)	Хэрэгжсэн

№	Шаардлага	Хэрэгжилт	Төлөв
6	Хүсэлтийг зөвхөн дараа нь систем safe хэвээр үлдэх үед олгох	`Banker.request`: Request ≤ Need → Request ≤ Available → түр олгох → safety → unsafe бол rollback	Хэрэгжсэн
7	Safe sequence гаргах	`Banker.is_safe` → (safe, sequence, steps)	Хэрэгжсэн
8	UI: Allocation, Max, Available-г өөрчилж, хүсэлт оруулан safe state шалгах	Засах боломжтой хүснэгт, хүсэлтийн форм, «▶ Шалгах»	Хэрэгжсэн
9	UI: процесс–нөөцийн холбоо ба safe sequence-ийг харуулах (Resource graph)	`graph.js` — SVG граф; ▶ Эхлүүлэх/Өмнөх/Дараах; Safety analysis хүснэгт	Хэрэгжсэн
10	Буруу/захын оролтыг шалгаж ойлгомжтой мэдээлэх (Guide §3.3)	`BankerError`, CLI-ийн `CliError`, HTTP 400, UI-д улаан нүд, идэвхгүй товч + мессеж	Хэрэгжсэн
11	Reset хийх боломж (Guide §5)	`Banker.reset`, UI «Анхны төлөв (Reset)»	Хэрэгжсэн
12	Manual-ын дагуу өөр компьютерт дахин ажиллуулах	`IV-2-13_Manual.pdf`, README Quick Start (гадны сангүй, зөвхөн Python 3)	Хэрэгжсэн

Номын шаардлагад байхгүй, манай багийн нэмсэн зүйлс: Banker-ыг унтрааж түгжрэлд хүрэхийг харуулах горим, хүсэлтийн түүх (лог), Allocation-ийг эхний төлвөөс өгөх сонголт, 3 бэлэн сценари (Unsafe, 10×5, Banker-гүй).

3. ОНОЛЫН ҮНДЭС

3.1 Түгжрэл ба Banker's Algorithm

Түгжрэл нь дөрвөн нөхцөл (mutual exclusion, hold and wait, no preemption, circular wait) зэрэг биелэхэд үүснэ. Banker's Algorithm нь түгжрэлээс *сэргийлэх* (deadlock avoidance) арга бөгөөд процесс бүр өөрийн нийт хэрэгцээг (Max) урьдчилан мэдэгдсэн байхыг шаарддаг. Систем хүсэлт бүрийг «түр олгоод» safe эсэхийг шалгаж, safe биш бол буцаан татгалзана.

3.2 Өгөгдлийн бүтэц

Хүснэгт 2. Banker's Algorithm-ын өгөгдлийн бүтэц

Нэр	Хэмжээс	Утга
Available	m	Одоо чөлөөтэй байгаа нөөц (нийт − Σ Allocation)
Max	n×m	Процесс бүрийн хамгийн их хэрэгцээ
Allocation	n×m	Процесс одоо барьж буй нөөц
Need	n×m	Need = Max − Allocation: дуустал нь дахин хэрэгтэй нөөц
Work, Finish	m, n	Safety algorithm-ын түр вектор ба «дууссан» тэмдэг

3.3 Safety algorithm

Код 1. Safety algorithm (сурах бичгийн дагуу)

```
1. Work = Available;  Finish[i] = false  (бүх i)
2. Finish[i] == false БА Need[i] <= Work байх i ол. Олдохгүй бол 4-рт.
3. Work = Work + Allocation[i];  Finish[i] = true;  2-рт буц.
4. Бүх i-д Finish[i] == true бол SAFE (олсон дараалал = safe sequence),
   үгүй бол UNSAFE.
```

Хугацааны нарийвчлал: гадна давталт ихдээ n удаа, дотор n процессыг m нөөцөөр харьцуулах тул $O(m \cdot n^2)$.

3.4 Resource-Request algorithm

P_i процесс Request[i] хүсэхэд:

- Request[i] ≤ Need[i] биш бол алдаа (процесс өөрийн Max-аас хэтэрсэн).
- Request[i] ≤ Available биш бол P_i хүлээнэ (нөөц хүрэлцэхгүй).
- Түр олгоно: Available −= Request; Allocation[i] += Request; Need[i] −= Request.
- Safety algorithm ажиллуулна.
- SAFE бол хүсэлтийг олгоно; UNSAFE бол түр өөрчлөлтийг буцааж (rollback) татгалзана.

3.5 SAFE, UNSAFE, DEADLOCK

SAFE төлөвт түгжрэл гарахгүй. UNSAFE төлөвт түгжрэл гарах *эрсдэл* бий боловч заавал гарахгүй. DEADLOCK бол процессууд бодитоор бие биенээ хүлээж гацсан төлөв. Banker's Algorithm нь системийг UNSAFE болгох хүсэлтийг татгалзаж, DEADLOCK-д хэзээ ч хүргэхгүй.

3.6 Гараар бодсон номын жишээ

5 процесс, нийт нөөц (10, 5, 7); одоогийн Available = (3, 3, 2). Need = Max − Allocation: P0 (7,4,3), P1 (1,2,2), P2 (6,0,0), P3 (0,1,1), P4 (4,3,1). Манай хэрэгжилт индексийн өсөх дарааллаар эхний тохирохыг сонгодог тул дараалал номынхоос өөр боловч хүчинтэй.

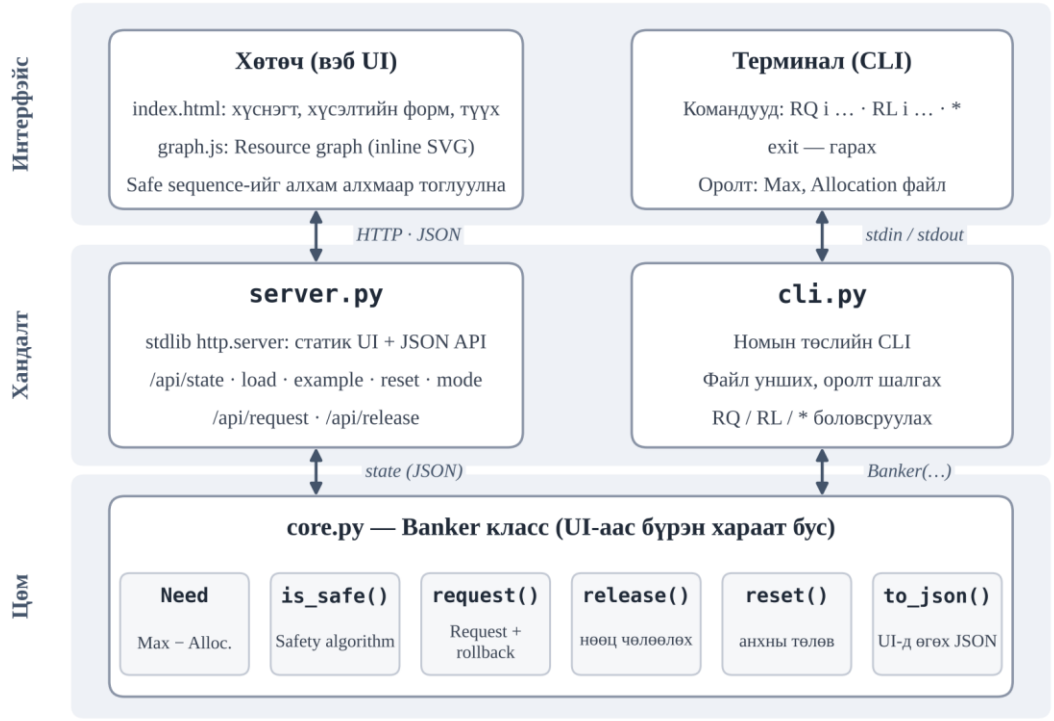
Хүснэгт 3. Safety algorithm-ын алхмууд (номын жишээ)

Алхам	Сонгосон процесс	Шалгалт Need ≤ Work	Work (шинэ)
0	—	—	(3, 3, 2)
1	P1	(1,2,2) ≤ (3,3,2) ✓	(3,3,2)+(2,0,0) = (5, 3, 2)
2	P3	(0,1,1) ≤ (5,3,2) ✓	(5,3,2)+(2,1,1) = (7, 4, 3)
3	P0	(7,4,3) ≤ (7,4,3) ✓	(7,4,3)+(0,1,0) = (7, 5, 3)
4	P2	(6,0,0) ≤ (7,5,3) ✓	(7,5,3)+(3,0,2) = (10, 5, 5)
5	P4	(4,3,1) ≤ (10,5,5) ✓	(10,5,5)+(0,0,2) = (10, 5, 7)

Дараалал <P1, P3, P0, P2, P4> нь SAFE. Номын <P1, P3, P4, P2, P0> дараалал ч хүчинтэй (тест `test_book_initial_safe` хоёуланг нь баталгаажуулдаг). Эцсийн Work нь нийт нөөцтэй (10, 5, 7) тэнцүү байна.

4. СИСТЕМИЙН ЗОХИОМЖ

Систем нь алгоритмын цөм, түүнийг ашигладаг хоёр гадаад интерфэйс (CLI, вэб) гэсэн давхартай. **Цөм нь UI-аас бүрэн хараат бус** тул UI-гүйгээр тестлэгддэг; UI нь алгоритмыг давхар хэрэгжүүлэлгүй, зөвхөн сервер (цөм) -өөс ирсэн JSON төлвийг зурдаг.



Зураг 1. Системийн зохиомж: интерфэйс, хандалт ба цөм

Хүснэгт 4. Эх кодын бүтэц

Файл	**Үүрэг**	**Мөр**
`src/core.py`	Banker класс: Need, Safety, Request (rollback), Release, Reset, to_json, оролтын шалгалт, enforce горим	184
`src/cli.py`	Номын төслийн командын мөрийн программ (RQ, RL, *, exit); файл унших, шалгах	186
`src/server.py`	stdlib `http.server`: статик UI + JSON API (state, load, example, reset, mode, request, release)	228
`src/ui/index.html`	UI: өгөгдлийн хүснэгт, хүсэлтийн форм, шалгалтын алхмууд, Safety analysis, түүх	560
`src/ui/graph.js`	Resource graph (inline SVG), тоглуулах удирдлага, hover/click тодруулалт	568
`tests/*.py`	5 файл, 64 автомат тест	~670

4.1 JSON гэрээ (core → UI)

Цөм `to_json()`-оор UI-д дараах бүтэц өгнө: `n, m, available, max, allocation, need, safe, sequence, steps[{process, work_before, work_after}]`. Safe биш үед `sequence` хоосон, `steps` нь зөвхөн дуусч чадсан процессуудыг агуулна. Дэлгэрэнгүй нь `docs/data-format.md`-д.

4.2 HTTP API

Хүснэгт 5. HTTP API

Зам	**Арга**	**Үүрэг**
`/api/state`	GET	Одоогийн төлөв
`/api/load`	POST	Шинэ өгөгдөл ачаалах (available, max, allocation) — буруу бол HTTP 400, төлөв өөрчлөгдөхгүй
`/api/example`	POST	Номын жишээг дахин ачаалах
`/api/reset`	POST	Сүүлд ачаалсан анхны төлөв рүү буцаах
`/api/mode`	POST	Banker-ыг асаах/унтраах (enforce)
`/api/request`, `/api/release`	POST	Хүсэлт, чөлөөлөлт: статус granted/wait/unsafe/error/released + мессеж

5. ХЭРЭГЖИЛТ

5.1 Хэл, орчин, сан

Python 3 (стандарт сан: `http.server`, `argparse`, `json`); UI нь цэвэр HTML/CSS/JavaScript, граф нь inline SVG — гадны сан, CDN ашиглаагүй тул интернетгүй ажиллана. Зөвхөн тест ажиллуулахад `pytest` хэрэгтэй. Build шаардлагагүй (интерпретатор).

5.2 Цөм (core.py)

`Banker` класс төлвийг (available, maximum, allocation) хадгалж, `need`-ийг property-оор тооцдог. Конструктор нь хэмжээс, бүхэл тоо, сөрөг утга, `Allocation > Max` зэргийг шалгаж `BankerError` шиднэ. `request()` нь буруу оролт (ERROR), нөөц хүрэлцэхгүй (WAIT), unsafe (UNSAFE, rollback), амжилттай (GRANTED) гэсэн статус буцааж, амжилтгүй үед төлвийг өөрчлөхгүй. Safety algorithm:

Код 2. Banker.is_safe() (src/core.py)

```
def is_safe(self):
    need = self.need; work = list(self.available)
    finish = [False] * self.n; sequence, steps = [], []
    while True:
        found = next((i for i in range(self.n) if not finish[i]
                      and all(need[i][j] <= work[j] for j in range(self.m))), None)
        if found is None: break
        before = list(work)
        work = [work[j] + self.allocation[found][j] for j in range(self.m)]
        finish[found] = True; sequence.append(found)
        steps.append({"process": found, "work_before": before, "work_after": list(work)})
    return all(finish), ..., steps
```

`valid_sequence()` нь өгсөн дараалал яг одоогийн төлөвт хүчинтэй эсэхийг алгоритмаас үл хамаарч шалгадаг тул тестэд «олон safe sequence байж болно» гэдгийг зөв тооцдог.

5.3 CLI (cli.py)

Номын шаардлагын дагуу нийт нөөцийг аргументаар, Max-ийг файлаар авч, `RQ i r0 r1 …`, `RL i r0 r1 …`, `*` командуудыг уншина. Файлын алдаа (олдохгүй, баганын тоо буруу, сөрөг/тоо биш утга, хоосон), Max > нийт нөөц гэх мэтийг ойлгомжтой мэдээлнэ.

5.4 Вэб UI

Хүснэгт ба хүсэлт. Available, Max, Allocation засах боломжтой; Need автоматаар; Allocation > Max үед Need улаан болж «Шалгах» товч идэвхгүй болно. Хүсэлтийн форм шалгалтын алхмуудыг (Request ≤ Need, ≤ Available, safety) ✓/✗-ээр харуулна. Түүх нь хүсэлт бүрийг дугаарлан үр дүн, шалтгаантай бүртгэнэ.

Resource graph (`graph.js`). Процесс — дугуй, нөөцийн төрөл — хүрээ, хүрээ доторх цэг — нөөцийн нэгж (процесст олгогдсон нь процессын өнгөөр, чөлөөтэй нь цагаан). Цэнхэр сум нөөц → процесс (Allocation), саарал тасархай сум процесс → нөөц (Need). «▶ Эхлүүлэх» дарахад процесс дуусах бүрд түүний сумнууд алга болж, нөөц чөлөөтэй болж, Work нэмэгдэнэ. Процесс эсвэл нөөц дээр hover/click хийхэд зөвхөн холбоотой

сумнууд тод харагдана. Граф нь серверээс ирсэн `steps`-ийг зурдаг тул алгоритмын бодит төлөвтэй 100% нийцнэ.

5.5 Бэлэн код, сан, AI-ийн хэрэглээ

Төслийн гол логик болон архитектурыг багийн гишүүд бие даан боловсруулж хэрэгжүүлсэн бөгөөд гадны ямар нэгэн хүндрүүлсэн сан ашиглаагүй (зөвхөн Python-ийн стандарт сан болон тестэд зориулж pytest, UI-д vis-network ашигласан). Алгоритмын суурь псевдокод нь сурах бичгийн 8-р бүлэгт бүрэн тулгуурласан болно. Хөгжүүлэлтийн явцыг хурдасгах болон дизайн, бүтцийг сайжруулах зорилгоор AI туслах хэрэгслүүдийг (Claude, Gemini) дараах байдлаар туслах чанарын түвшинд ашигласан: Core алгоритм болон автомат тестийн эхний бүтцийг гаргахад санаа авах; UI интерфейсийн layout, харагдах байдал, хэрэглэгчийн туршлагыг (UX) илүү ойлгомжтой болгож сайжруулах. AI-аас гарсан санаа, кодыг багийн гишүүд нэг бүрчлэн нягтлан шалгаж, өөрсдийн логикоор оновчлон, автомат тестүүдийг бүрэн ажиллуулж баталгаажуулсан болно

5.6 GitHub

Repository: `https://github.com/jal0115/os-bd1-IV-2-13-bankers-algorithm`. Бүтэц: `README.md, src/, tests/, examples/, docs/`. Ажил PR-аар (жишээ нь UI — PR #6, Resource graph — feature branch) main-д нэгдсэн; 9-р долоо хоногт Release/Tag `v1.0` нийтэлнэ. Нууц түлхүүр, хувийн өгөгдөл оруулаагүй.

6. ТУРШИЛТ БА UI ДҮРСЛЭЛ

6.1 Юуг өөрчилж турших вэ

- Бэлэн сценари (dropdown): Номын жишээ (5×3), Unsafe эхний төлөв (3×2), Том туршилт (10×5), Banker-гүй горим.
- Хүснэгтийн тоо: Available, Max, Allocation-г шууд засах; +/− Процесс, Нөөц-өөр хэмжээг өөрчлөх.
- Хүсэлт/Release: аль ч процесс, аль ч хэмжээгээр.
- Banker асаалттай/унтраалттай горим.
- Reset: ачаалсан анхны төлөв рүү буцах.

6.2 Юуг ажиглах вэ

- Дээд баджид SAFE/UNSAFE, «Аюулгүй дараалал» карт.
- Шалгалтын алхмууд: ✓ Request ≤ Need, ✓ Request ≤ Available, ✓/✗ safety.
- Resource graph: сумны тоо = Allocation/Need, нөөцийн хүрээн дэх чөлөөтэй цэгийн тоо = Available/Work. Дууссан процесс бүдгэрч ✓ тэмдэгтэй болно.
- Safety analysis хүснэгт: алхам бүрийн Work (өмнө) → Work (дараа).

6.3 Туршилтын сценарууд

Хүснэгт 6. Туршилтын сценарууд

№	Сценари	Banker	Юуг харуулах
1	Номын жишээ (5×3), хүсэлтүүд	Асаалттай	P1 (1,0,2) олгогдоно; P4 (3,3,0) нөөц хүрэхгүй тул хүлээнэ; P0 (0,2,0) нөөц байгаа ч UNSAFE болох тул татгалзана
2	Unsafe эхний төлөв (3 процесс, 2 нөөц)	Асаалттай	Эхнээсээ UNSAFE; P0-ийн Release (2,1)-ээс хойш SAFE <P1,P0,P2>
3	Том туршилт 10×5	Асаалттай	Олон процесс, нөөц; нөөцийн нэгж туузаар зурагдана; граф уншигдах чанар
4	Банкиргүй: Deadlock-д хүрэх	Унтраалттай	P1 (1,0,2), дараа нь P0 (0,2,0) хоёулаа олгогдож төлөв UNSAFE болно (Banker-тэй бол P0 татгалзагдана)
5	Алдаатай оролт	—	Need-ээс хэтэрсэн хүсэлт, Allocation > Max (Need сөрөг, товч идэвхгүй)

7. ТУРШИЛТЫН АРГА

Хүснэгт 7. Туршилтын арга

Зүйл	Тодорхойлолт
Орчин	Ubuntu 24.04.4 LTS (x86_64), Python 3.12.3, Chromium (Playwright 1.56, headless) — screenshot авсан орчин. Windows дээрх туршилтыг Хавсралт Б-д нөхөж бичнэ.
Өгөгдөл	Номын 8.6.3 жишээ (`examples/book_*.txt`), UI-ийн 4 сценари, тестийн кодод тодорхойлсон санамсаргүй өгөгдөл (seed тогтмол: 2026, 7)
Автомат тест	`python -m pytest -q` → 64 тест (`tests/`)
Бие даасан шалгагч	`valid_sequence()` нь олдсон дарааллыг алгоритмаас үл хамааран шалгана; `brute_safe` нь бүх n! сэлгэмэлийг туршина (200 санамсаргүй 5×3 төлөв дээр цөмийн safe/unsafe-тай таарна)
Хүлээгдсэн үр дүн	Номын жишээний SAFE/UNSAFE, олгох/хүлээх/татгалзах; инвариант: «татгалзсан хүсэлт төлвийг өөрчлөхгүй (rollback)»; буруу оролтод ERROR/HTTP 400 ба төлөв өөрчлөгдөхгүй
Дахин шалгах	Repository clone → Manual-ын 2-р хэсэг → `python -m pytest -q`

8. ҮР ДҮН

8.1 Автомат тестийн үр дүн

Хүснэгт 8. Автомат тестийн үр дүн

Файл	Тест	Тэнцсэн	Хамрах зүйл
`tests/test_core.py`	15	15	Номын жишээ (Need, safe, дараалсан хүсэлт), T1 Request>Need, T2 буруу оролт, T3 Unsafe эхний төлөв, T4 Release-ээс хойш safe, T5 10×5, brute-force тулгалт, T6 Reset, конструкторын шалгалт, to_json
`tests/test_cli.py`	24	24	RQ/RL/*, экспортын алдаа, файлын алдаа (олдохгүй, баганын тоо, сөрөг, хоосон), --allocation, 4 төрлийн нөөц, коммент/хоосон зай
`tests/test_cli_state_integrity.py`	7	7	Алдаатай/татгалзсан хүсэлтийн дараа төлөв хэвээр, request→release буцаалт
`tests/test_server.py`	13	13	HTTP: эхний төлөв, номын дараалал, алдаа, load/reset, буруу JSON, статик файл, path traversal, unicode мессеж
`tests/test_banker_off.py`	5	5	Banker унтраалттай горим, /api/mode
Нийт	**64**	**64**	0 унасан

8.2 Номын жишээний дараалсан хүсэлтүүд (бодит гаралт)

CLI-аар (`examples/book_commands.txt`) гарсан үр дүн:

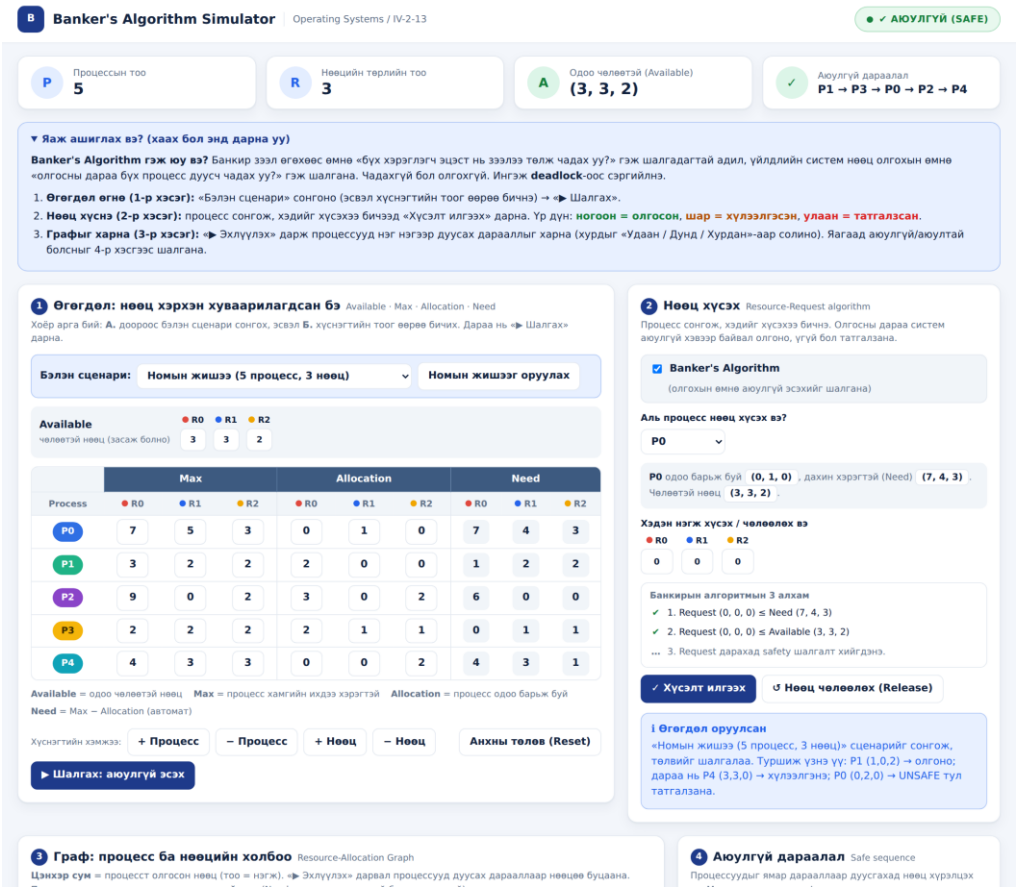
Код 3. Номын жишээний CLI гаралт

```
$ python src/cli.py 10 5 7 --max examples/book_max.txt \
      --allocation examples/book_allocation.txt < examples/book_commands.txt
Available: 3 3 2          ...  Төлөв: SAFE  <P1, P3, P0, P2, P4>
granted: P1-д олголоо; төлөв safe хэвээр                    (Available -> 2 3 0)
wait:    P4 хүлээнэ: R0 хүсэлт=3, Available=2
unsafe:  P0-ийн хүсэлтийг олговол unsafe болно; татгалзлаа     (төлөв өөрчлөгдсөнгүй)
```

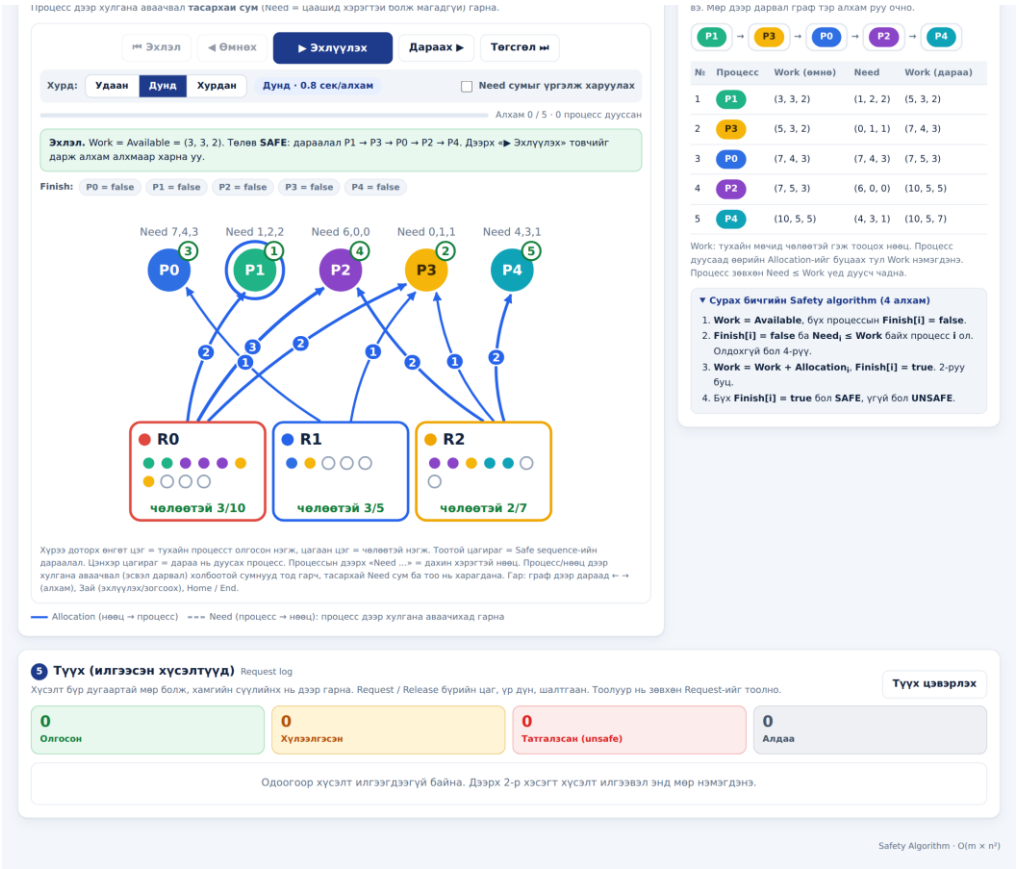
Хүснэгт 9. Номын жишээний дараалсан хүсэлтүүд

Дараалал	Хүсэлт	Шалгалт	Үр дүн	Available
1	P1 (1,0,2)	≤ Need (1,2,2) ✓; ≤ Available (3,3,2) ✓; түр олгоод safety → safe	Олгосон	(3,3,2) → (2,3,0)
2	P4 (3,3,0)	≤ Need (4,3,1) ✓; R0: 3 > Available 2 ✗	Хүлээнэ	(2,3,0) хэвээр
3	P0 (0,2,0)	≤ Need (7,4,3) ✓; ≤ Available ✓; түр олгоход Available (2,1,0) → ямар ч процесс дуусахгүй → unsafe	Татгалзсан (rollback)	(2,3,0) хэвээр

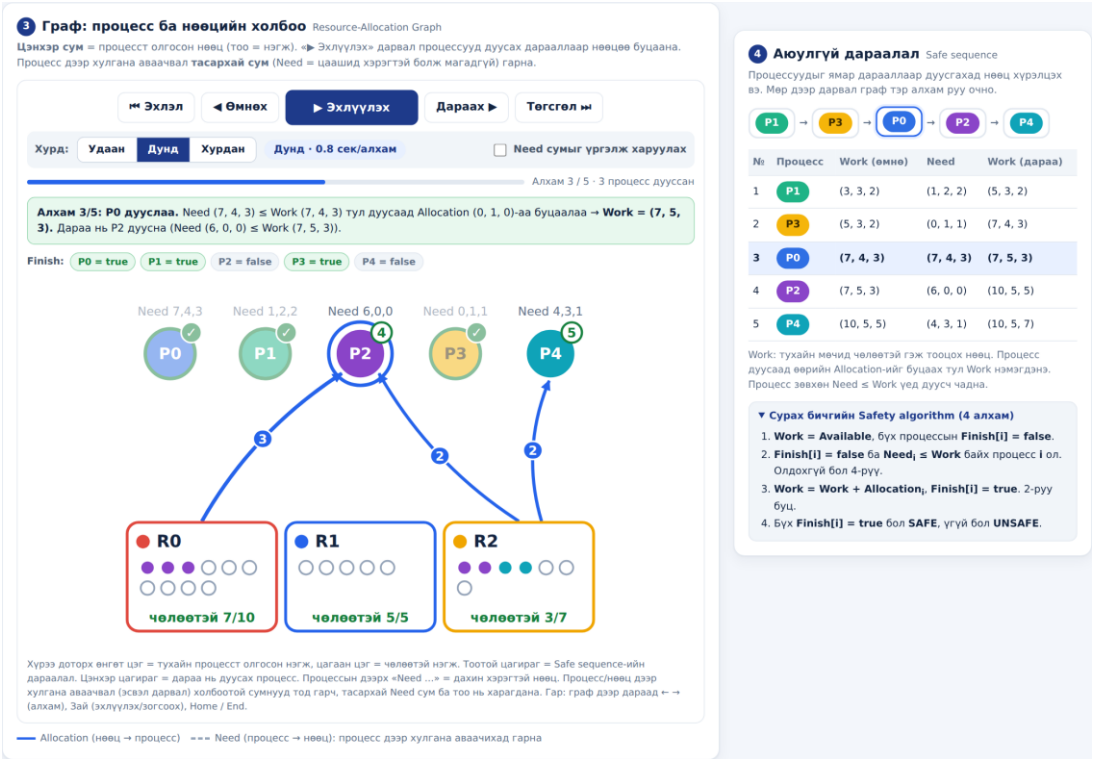
8.3 Бодит screenshot (UI)



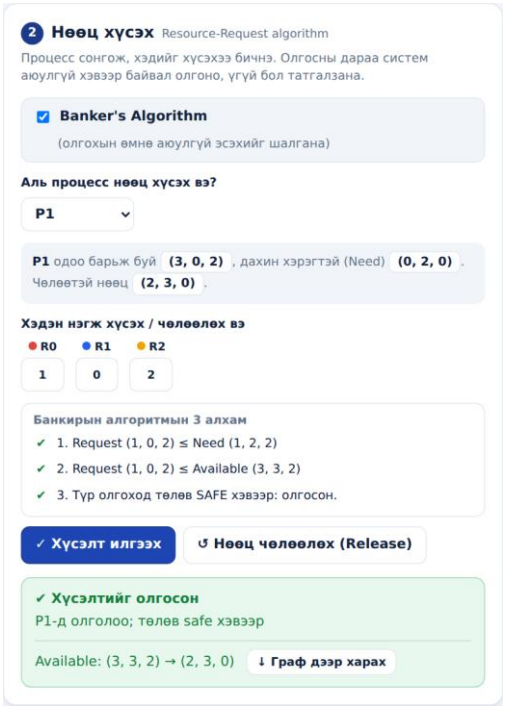
Зураг 2. Програмын нээгдсэн байдал (дээд хэсэг): толгой, «Яаж ашиглах вэ?», (1) өгөгдлийн хүснэгт, (2) хүсэлт.



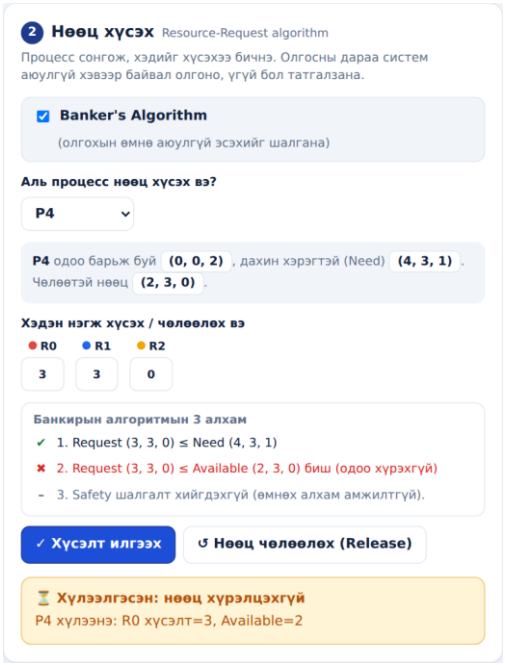
Зураг 3. Програмын нээгдсэн байдал (доод хэсэг): (3) Resource graph, (4) Аюулгүй дараалал, (5) түгх.



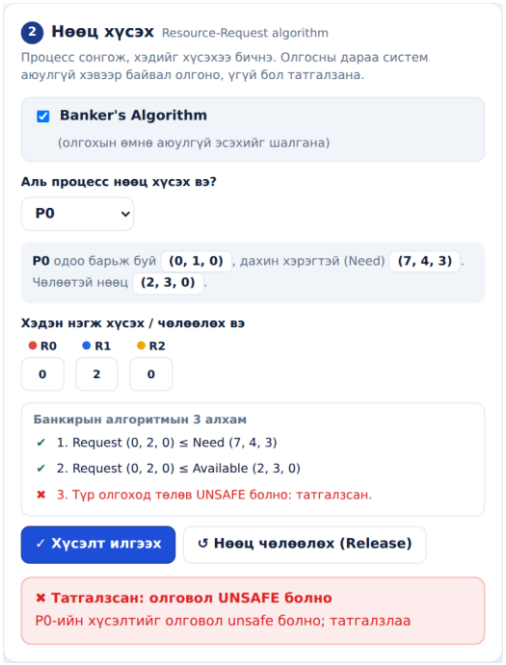
Зураг 4. Safe sequence-ийн 3-р алхам: P1, P3 дууссан (бүдэг, ✓), P0 дараа нь дуусна (Work = (7,4,3)).



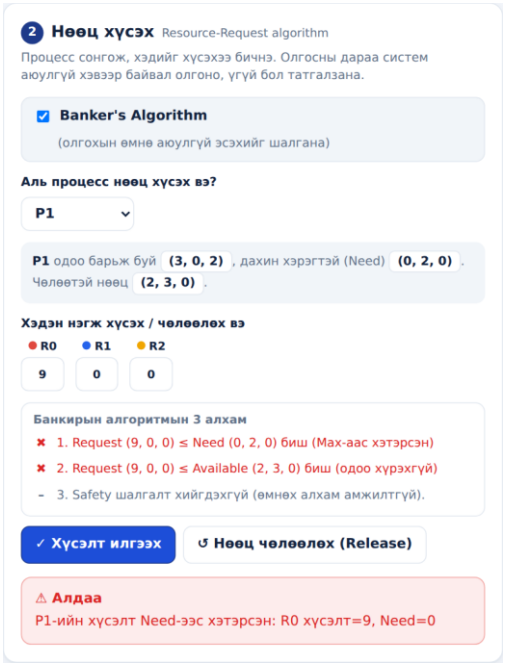
Зураг 5. P1 (1,0,2) хүсэлт олгогдов: гурван шалгалт ✓.



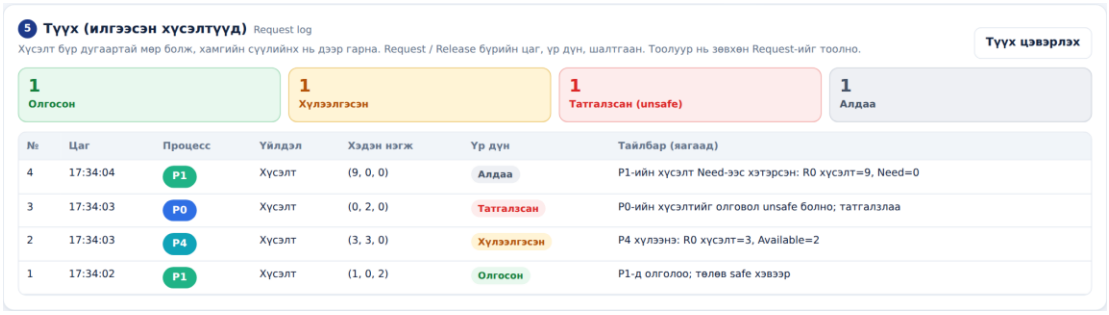
Зураг 6. P4 (3,3,0): R0 нөөц хүрэхгүй тул хүлээнэ.



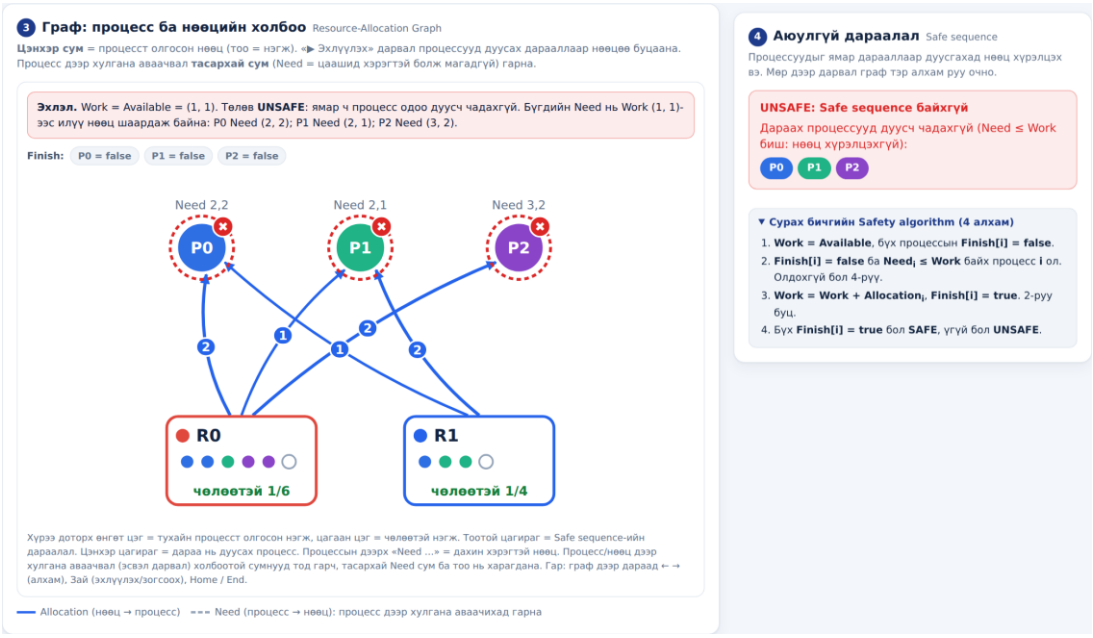
Зураг 7. P0 (0,2,0): Need, Available ✓ боловч түр олговол UNSAFE тул татгалзав.



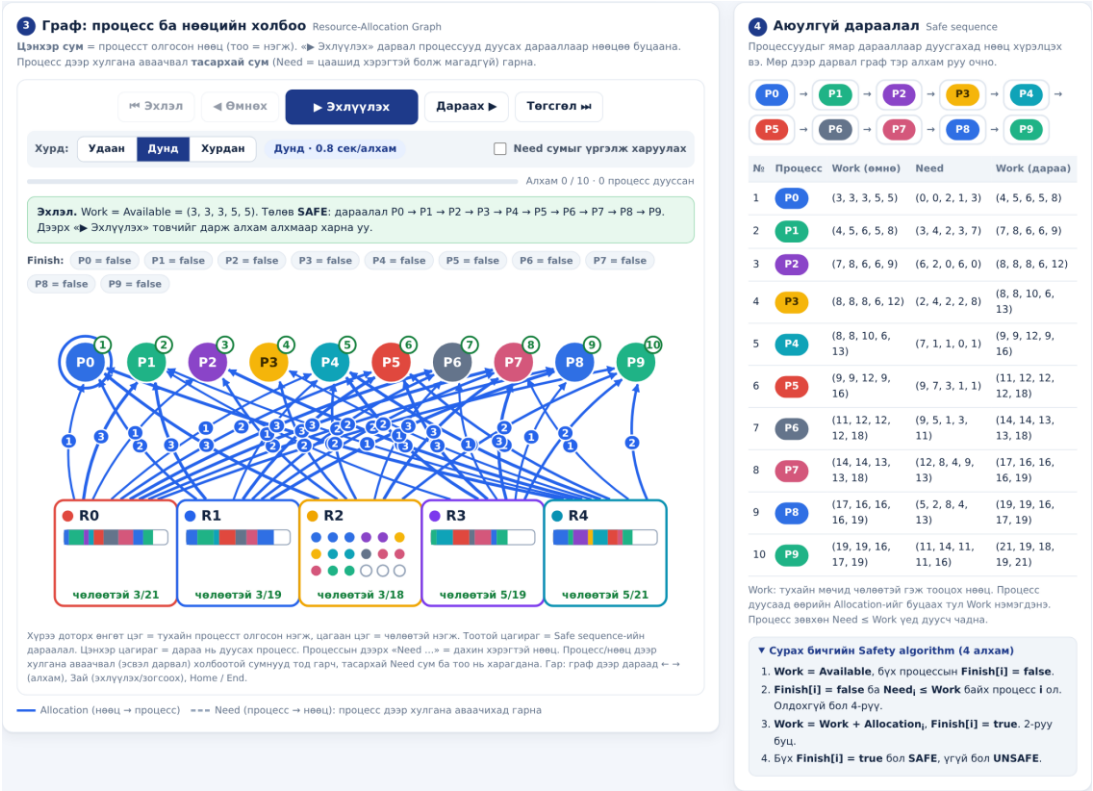
Зураг 8. Need-ээс хэтэрсэн хүсэлт (9,0,0): ERROR, төлөв өөрчлөгдөөгүй.



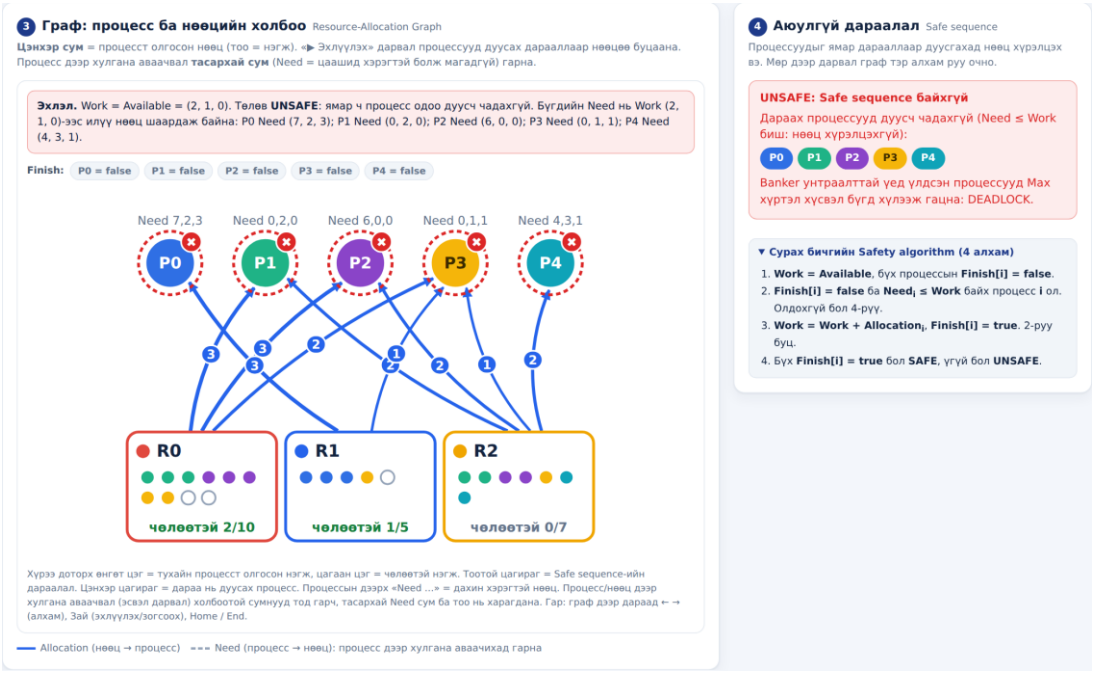
Зураг 9. Түүх: хүсэлт бүр дугаартай, үр дүн ба шалтгаантай; тоолуур.



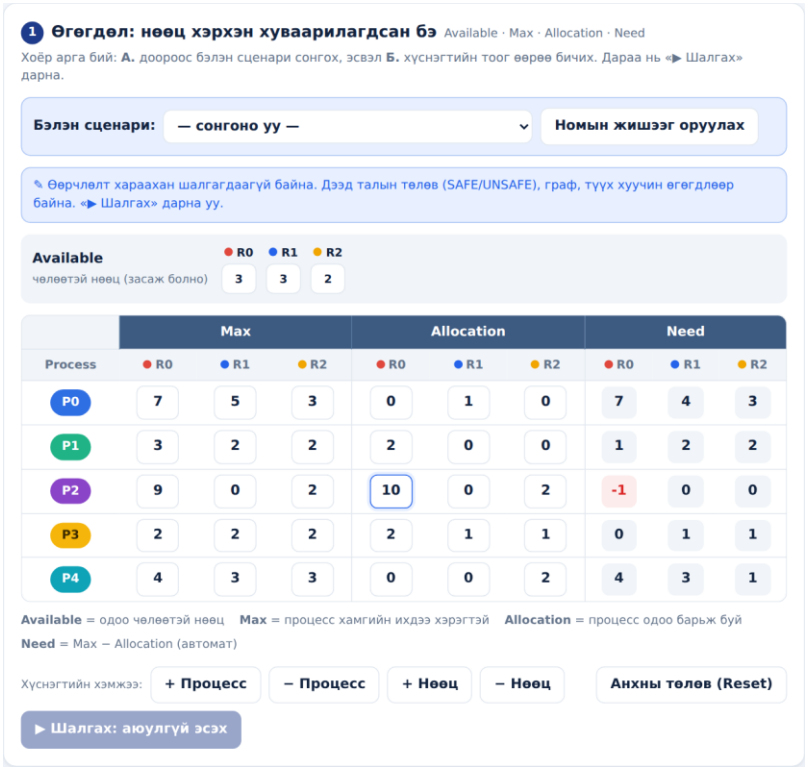
Зураг 10. Unsafe эхний төлөв: ямар ч процесс дуусч чадахгүй (улаан ✕), Safe sequence байхгүй.



Зураг 11. Том туршилт 10×5: нөөцийн олон нэгж туузаар зурагдана; процесс дээр дарж холбоотой сумыг тодруулна.



Зураг 12. Banker унтраалттай: P1, дараа нь P0 хоёулаа олгогдож төлөв UNSAFE болсон.



Зураг 13. Оролтын алдаа: P2-ийн Allocation (10) > Max (9) → Need = −1 (улаан), «Шалгах» товч идэвхгүй.

8.4 Нэмэлт туршилтын дүгнэлт

Хүснэгт 10. Нэмэлт туршилтууд

Туршилт	Ролт	Хүлээгдсэн	Бодит
Request > Need	P1 (2,0,0), эхний төлөв	ERROR, төлөв хэвээр	ERROR (`test_t1`)
Буруу ролт	(−1,0,2), урт буруу, тэмдэгт, 1.5, True, процесс 9/−1	ERROR	ERROR ×7 (`test_t2`)
Unsafe эхний төлөв	Available (2,1,0), тестийн Allocation	UNSAFE, дараалал хоосон	UNSAFE; brute-force ч unsafe
Release-ээс хойш safe	Дээрхээс P4 (0,0,2) чөлөөлөх	Available (2,1,2), SAFE	SAFE, хүчинтэй дараалал
Банкиргүй	P1 (1,0,2), P0 (0,2,0)	Хоёулаа олгогдож UNSAFE	GRANTED, GRANTED, unsafe (`test_banker_off`)
Reset	Хүсэлт, release-ийн дараа reset	Анхны төлөв	Нийцсэн (`test_t6`)
Файлын алдаа (CLI)	Олдохгүй файл, баганын тоо буруу, хоосон Max	Ойлгомжтой алдаа	Нийцсэн (`test_cli`)

9. ХЭЛЭЛЦҮҮЛЭГ

9.1 Үр дүнгийн тайлбар

- Нөөц хүрэлцэх нь хүсэлтийг олгох хангалттай нөхцөл биш: P0 (0,2,0) нөөц байсан ч олгогдсонгүй, учир нь түүний дараа Available (2,1,0) болж ямар ч процесс дууссангүй (UNSAFE). Banker унтраалттай сценариор үүнийг бодитоор харуулсан.
- Эхний төлөвт P4 (3,3,0) нь Available (3,3,2)-д багтдаг тул WAIT биш (`test_p4_on_fresh_state_is_not_wait`); номын «P4 хүлээнэ» нь P1-ийн хүсэлт олгогдсоны дараах төлөвт хамаарна.
- Safe sequence ганц биш. Манай хэрэгжилт индексийн дарааллаар эхний тохирохыг авдаг тул <P1,P3,P0,P2,P4> гарч, номын <P1,P3,P4,P2,P0>-аас ялгаатай ч `valid_sequence`-ээр хоёулаа хүчинтэй.

9.2 Хязгаарлалт ба мэдэгдэж буй асуудлууд

- **Нэг хэрэглэгчийн төлөв.** Сервер төлвийг санах ойд, нэг хуваалцсан хэлбэрээр хадгална; олон хөтөч/таб нэг төлөвийг хуваалцана, сервер дахин асаахад төлөв анхны жишээ рүү буцна.
- **Олон thread-тэй customer.** Номын төслийн сонголттой хэсэг (mutex-тэй олон thread) хэрэгжээгүй; хүсэлтүүд дарааллаар боловсруулагдана.
- **10×5 дээрх граф.** Сум олон тул давхцаж харагдана; hover/click-ээр холбоотой сумыг тодруулж хөнгөвчилсөн.
- **Нэг сервер.** Нэмэлт аюулгүй байдлын хамгаалалт (нэвтрэлт) байхгүй; зөвхөн 127.0.0.1 дээр асна.
- **Орчин.** Автомат screenshot нь Linux дээр авсан; Windows/macOS дээр гараар шалгахыг Хавсралт Б-д бичнэ.

10. ДҮГНЭЛТ

Төсөл нь сурах бичгийн Banker's Algorithm-ын шаардлагыг (Available аргументаар, Max файлаас, `RQ/RL/*`, safety-тай хүсэлт, rollback) бүрэн хэрэгжүүлж, багшийн заасан Resource graph дүрслэлийг бодит төлөвтэй холбон алхам алхмаар тоглуулдаг интерактив UI болгосон. 64 автомат тест тэнцэж, номын жишээ, Unsafe, 10×5, Banker-гүй, алдаатай оролтын сценаруудыг UI дээр бодитоор баталгаажуулсан. Хэрэглэгч Allocation, Max, Available, хүсэлтийг өөрчилж, Reset хийж дахин турших боломжтой. Цаашид олон thread-тэй customer, төлвийг хөтөч бүрээр тусгаарлах, граф дахь сумны давхцлыг багасгах ажлыг хийж болно.

АШИГЛАСАН МАТЕРИАЛ

- A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. 8-р бүлэг: Deadlocks (Banker's Algorithm, Programming Projects).
- E. W. Dijkstra. Banker's algorithm-ын анхны санаа (EWD108), 1965.
- Python 3 Documentation: `http.server`, `argparse`.
- MDN Web Docs: SVG, DOM API.
- pytest documentation.
- Төслийн эх код: https://github.com/jal0115/os-bd1-IV-2-13-bankers-algorithm
- AI (Claude, Gemini): 5.5 хэсэгт заасан хүрээнд.

ХАВСРАЛТ А. АЖЛЫН ТӨЛӨВЛӨГӨӨ

Хүснэгт 11. Төслийн мэдээлэл

Талбар	**Мэдээлэл**
Хуваарилалтын код	IV-2-13
Төсөл	Banker's Algorithm (Resource graph UI)
Repository	`https://github.com/jal0115/os-bd1-IV-2-13-bankers-algorithm`
Хэл/сан/орчин	Python 3 (стандарт сан), HTML/CSS/JavaScript, inline SVG; тест: pytest
Үндсэн дүрслэл	Resource graph (процесс–нөөцийн граф, Safe sequence алхамчилсан)

Гишүүн бүрийн хариуцсан ажил

Хүснэгт 12. Гишүүдийн хариуцсан ажил

№	**Гишүүн (үүрэг)**	**Хариуцсан ажил ба хүрэх үр дүн**	**Commit/file**
1	Г.Лувсанжал /B242270016/	Цөм алгоритм ба CLI: Need тооцоо, Safety algorithm, Resource-Request algorithm, rollback, CLI болон файл унших суурь логик.	src/core.py, src/cli.py, tests/
2	Г.Ганжаргал /B222270701/	CLI команд болон файл унших хэсгийн анхны бүтцэд санаачилга гаргаж оролцсон.	src/cli.py
3	Б.Лхагвадорж /B231940136/	Вэб UI-ийн хүснэгт болон анхны загвар зохион байгуулалтад тусалсан.	src/ui/index.html
4	У.Анужин /B241940071/	UI интерфейс, Граф ба Туршилт: Засах боломжтой хүснэгт, Request/Release форм, Resource graph (inline SVG), Safe sequence алхамчилсан төлөв, UI/UX сайжруулалт, нэмэлт тестүүд.	src/ui/index.html, src/ui/graph.js
5	Я.Төгөлдөр /B242270806/	Safe sequence ба төлөвийн логикийг шалгахад хамтарсан.	src/ui/
6	Г.Ганхүү /B210930028/	Docs ба README: Төслийн баримт бичиг (Docs), хэрэглэгчийн болон ашиглалтын заавар (Manual), мөн README.md файлуудыг боловсруулж шинэ орчинд ажиллах нөхцөлийг бүрдүүлсэн.	docs/, README.md

Тестийн төлөвлөгөө

Хүснэгт 13. Тестийн төлөвлөгөө

Тест	**Оролт/нөхцөл**	**Хүлээгдсэн үр дүн**
Номын жишээ	Available (3,3,2)	SAFE; P1 олгоно; P4 хүлээнэ; P0 татгалзана
Unsafe → Release	Unsafe эхний төлөв, P4 (0,0,2) чөлөөлөх	UNSAFE → SAFE
Захын/буруу оролт	−1, урт буруу, тэмдэгт, Need-ээс хэтэрсэн, процесс 9	ERROR, төлөв хэвээр
10×5 том өгөгдөл	seed 2026, 50 төлөв	Дараалал хүчинтэй, эцсийн Work = нийт нөөц
Brute-force тулгалт	200 санамсаргүй 5×3	safe/unsafe таарна
Banker-гүй	P1 (1,0,2), P0 (0,2,0)	Хоёул олгогдож UNSAFE

ХАВСРАЛТ Б. НЭМЭЛТ ОРЧИНД ШАЛГАСАН ТЭМДЭГЛЭЛ

Манай баг дараах орчинд Manual-ын дагуу Build/Run, `python -m pytest -q` хийж баталгаажуулна (бөглөх):

Хүснэгт 14. Нэмэлт орчинд шалгасан тэмдэглэл

OS	**Python**	**pytest үр дүн**	**UI (хөтөч)**
Ubuntu 24.04.4 (x86_64)	3.12.3	64 тест (шалгагч скриптээр 64/64)	Firefox
[Windows 10/11]	3.12.3	64 passed (7.74s)	Chrome
[macOS]	[хувилбар]	[]	[]